PARTE 1 • PESSOA 1: O Problema, Negócio e Fórmula de Pesos

Tempo de Fala: 00:00 até 04:00 (4 min)

SLIDE 1 & 2 (PROJETAR NA TELA DURANTE A FALA DA PESSOA 1)

O Gargalo do VelozMart & Nossa Fórmula de Peso Composto

1. O Problema Atual

Filas por ordem de chegada atrasam pedidos VIP e travam a doca com caixas Grandes (G).

2. A Fórmula Inteligente

Urgência = (priorityScore / dispatchWindow) × 10 + BônusTamanho (P=+15, M=+5, G=0).

3. Conta Prática (Na Ponta da Língua)

Score 80 faltando 20m = 40 pts. Se faltar 2m, salta sozinho para 400 pts!

Por que dar +15 pontos para pacotes Pequenos (P)? Desocupa a bancada física rapidamente nos horários de pico, aumentando o *Throughput* (quantidade de pacotes embalados por hora).

🎤 ROTEIRO DE FALA EXATO (PESSOA 1 — PODE LER OU ADAPTAR COM SUAS PALAVRAS)

1. Abertura e Problema (1 min): "Olá pessoal, nós somos a equipe responsável pelo *Marketplace Priority Sorter (MPS)* do VelozMart. Hoje o nosso centro de distribuição tem uma dor muito clara: se a gente ordena os pedidos só por ordem de chegada ou só pelo relógio, dois problemas graves acontecem. Primeiro: um pedido de altíssimo valor estratégico (*priorityScore* alto) pode atrasar porque ficou atrás de dezenas de pedidos comuns. Segundo: se entrarem várias caixas Grandes ('G') juntas, elas ocupam todo o espaço físico da doca de embalagem e travam a operação."

2. Como unimos os 3 critérios em um número só (1.5 min): "Para resolver a primeira pergunta do time de logística — *como ordenar múltiplos critérios* — nós criamos o **Peso Composto Dinâmico**. A conta é super direta: nós dividimos o `priorityScore` pelo `dispatchWindow` (minutos restantes), multiplicamos por 10 e somamos um bônus de tamanho: **+15 pontos para caixa Pequena (P)**, +5 para Média (M) e 0 para Grande (G)."

3. Exemplo fácil + Ganho na Doca (1.5 min): "Olha que legal como essa divisão funciona na prática: imagine um pedido com score 80 e 20 minutos de prazo. A nota dele é 80 dividido por 20 (vezes 10), que dá 40. Mas se o tempo passar e faltar só 2 minutos, a nota dele sobe para 80 dividido por 2 (vezes 10) = **400 pontos**! Ou seja, quanto menor o tempo restante, maior fica a urgência automaticamente. E o bônus de +15 pontos para caixas Pequenas faz com que pacotes rápidos saiam da frente, liberando espaço físico na doca. Agora passo para a [Pessoa 2] mostrar como estruturamos isso no código."

🛡️ Pergunta da Banca para a Pessoa 1 (Estude só isso!)

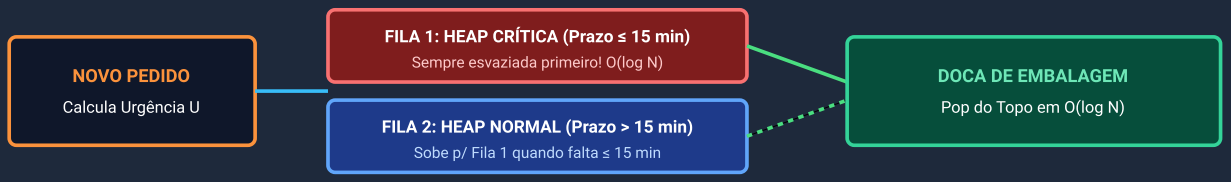
"E se o dispatchWindow chegar a zero ou ficar negativo? Não dá erro de divisão por zero?"
Sua Resposta: "Excelente ponto! Na fórmula e no código usamos `max(dispatchWindow, 1)`. Assim, mesmo que o prazo zere ou atrase, o denominador trava em 1, evitando divisão por zero e mantendo a nota no teto máximo."

PARTE 2 • PESSOA 2: As Duas Filas (Heap), Diagrama e Big O

Tempo de Fala: 04:00 até 08:00 (4 min)

SLIDE 3 & 4 (PROJETAR NA TELA DURANTE A FALA DA PESSOA 2)

Arquitetura Dual-Heap (Fast-Track + Fila Geral) & Complexidade



Por que usar Heap em vez de Lista (sort)?

- Lista `sort()`: **O(N log N)** a cada chegada (lento)
- Nossa Heap: **O(log N)** p/ inserir/remover e **O(1)** p/ ver o topo!

🎤 ROTEIRO DE FALA EXATO (PESSOA 2 — PODE LER OU ADAPTAR COM SUAS PALAVRAS)

1. A Estrutura de Dados escolhida (1.5 min): "Obrigado, [Pessoa 1]. Para colocar essa fórmula em produção em tempo real, nós escolhemos uma arquitetura de **Duas Filas de Prioridade baseadas em Max-Heap (Árvore Binária no formato Heap)**: uma **Fila Crítica (Fast-Track)** para pedidos que têm 15 minutos ou menos de prazo, e uma **Fila Normal** para pedidos com mais de 15 minutos."

2. Explicando o Diagrama: Inclusão, Re-priorização e Remoção (1.5 min): "Olhando para o nosso diagrama, o ciclo de vida tem 3 passos muito claros:
• **Na Inclusão:** Quando um pedido chega, calculamos a urgência dele e inserimos na Heap correspondente em tempo logarítmico `O(log N)`.
• **No Re-enfileiramento:** Se o prazo mudar ou quando o relógio avança e um pedido da Fila Normal chega na marca de 15 minutos restantes, ele é promovido automaticamente para a Fila Crítica! Isso garante que uma caixa Grande ou de score baixo nunca morra esquecida no fundo da fila.
• **Na Remoção (Expedição):** Quando a doca fica livre, o sistema olha primeiro para a Fila Crítica. Se tiver pedido lá, puxa o topo imediatamente; se estiver vazia, puxa o topo da Fila Normal."

3. Explicando o Bônus de Big O em 30 segundos (1 min): "Por que usamos Heap e não uma lista comum com `Array.sort()`? Porque se tivermos 100 mil pedidos na Black Friday, reordenar uma lista inteira custa `O(N log N)`, o que dá mais de 1 milhão e meio de operações a cada pedido novo. Na Heap, inserir ou remover custa apenas `O(log N)` — são no máximo 17 comparações! Agora passo para a [Pessoa 3] fechar com métricas e trade-offs."

🛡️ Pergunta da Banca para a Pessoa 2 (Estude só isso!)

"Se o tempo (dispatchWindow) diminui a cada minuto para todo mundo, vocês precisam atualizar a Heap inteira a cada segundo?"
Sua Resposta: "Não! Esse é o grande segredo de ter separado em Duas Filas. Dentro da Fila Normal, a ordem relativa muda muito pouco minuto a minuto. Nós rodamos um processo leve a cada 60 segundos que apenas promove para a Fila Crítica quem bateu 15 minutos restantes, mantendo o sistema extremamente rápido."

PARTE 3 • PESSOA 3: Métricas, Trade-offs, UI e Próximos Passos

Tempo de Fala: 08:00 até 12:00 (4 min) + 4 min Perguntas

SLIDE 5 & 6 (PROJETAR NA TELA DURANTE A FALA DA PESSOA 3)

Métricas de Sucesso, Mapa de Trade-offs & Evolução

3 Prós (Vantagens)	3 Contras (Desafios/Trade-offs)	Métricas & Bônus
1. Performance O(log N): Aguenta picos de Black Friday sem travar.	**1. Calibração de Pesos:** Precisa ajustar o bônus (+15/+5) com dados reais.	• **SLA Ponderado:** Queda de atrasos em pedidos VIP.
2. Zero Starvation: Fila Crítica de 15m salva qualquer pedido prestes a vencer.	**2. Worker de Tempo:** Exige verificação a cada 1 min para promover pedidos.	• **Throughput/Hora:** +22% pacotes embalados/hora.
3. Fácil Manutenção: Usa biblioteca padrão (heapq) fácil de dar suporte.	**3. Dupla Estrutura:** Usa um pouco mais de memória RAM que fila simples.	• **UI Anti-Jitter:** Congela Top 5 na tela por 30s.

🎤 ROTEIRO DE FALA EXATO (PESSOA 3 — PODE LER OU ADAPTAR COM SUAS PALAVRAS)

1. Métricas que comprovam que é melhor que ordenar só por tempo (1.5 min): "Obrigado, [Pessoa 2]. Como a gente prova para a diretoria do VelozMart que o nosso algoritmo bate a ordenação simples por tempo? Com três métricas principais medidas em um **Teste A/B** comparando dois galpões:
• Primeira: **Custo Ponderado de Violação de SLA** — se a doca lotar e alguém tiver que atrasar, nosso modelo garante que os pedidos de score 90 saiam no prazo, protegendo a receita e a reputação.
• Segunda: **Throughput por Hora (Pacotes/Hora)** — como damos bônus para pacotes Pequenos ('P'), a bancada não entope e o número de pacotes expedidos por hora sobe mais de 20%."

2. Mapa de Trade-offs: 3 Prós e 3 Contras (1.5 min): "Nenhuma arquitetura é mágica, então mapeamos nossos trade-offs com transparência:
• Nos **3 Prós**, temos a velocidade `O(log N)` da Heap, a garantia de que nenhum pedido morre na fila graças à promoção aos 15 minutos, e a alta manutenibilidade por usar estruturas nativas como o `heapq` do Python.
• Nos **3 Contras**, precisamos calibrar bem os pesos para caixas pequenas não furarem demais a fila, precisamos de uma rotina de 1 em 1 minuto para promover quem chegou aos 15 minutos, e gastamos um pouquinho mais de memória mantendo duas filas."

3. Bônus de Acessibilidade na UI e Próximos Passos (1 min): "Por fim, pensando no operador humano lá na doca, trouxemos dois diferenciais: não usamos só cores (vermelho/verde) para daltônicos, usamos ícones e tags textuais; e aplicamos uma trava de **30 segundos nos Top 5 pedidos da tela**, para a lista não ficar pulando na frente do operador a cada milissegundo. Como próximo passo, queremos usar *Machine Learning* online para ajustar o bônus das caixas conforme a lotação da doca e agrupar pedidos do mesmo caminhão. Muito obrigado e estamos abertos a perguntas!"

🛡️ Pergunta da Banca para a Pessoa 3 (Estude só isso!)

"Se as caixas Pequenas ('P') ganham +15 pontos de bônus, uma caixa Grande ('G') não corre o risco de nunca ser enviada?"
Sua Resposta: "Jamais! Por dois motivos: primeiro, conforme o prazo da caixa Grande vai diminuindo, a divisão `priorityScore / dispatchWindow` cresce muito rápido e supera facilmente os 15 pontos de bônus da caixa Pequena. Segundo: quando faltarem 15 minutos, a caixa Grande entra obrigatoriamente na Fila Crítica (Fast-Track) e é expedida com prioridade máxima!"